

Neural Network Optimization (Report)

Loading the required Libraries and the Dataset

As an initial step to complete the tasks, all the required libraries are imported.

After importing the libraries and the dataset, the pixel values are normalized by dividing them by 255, to be in a range [0,1]. This makes the weights with small values, which doesn't slow down the learning process in Neural Networks.

Encoding the labels (output) into one-hot vector, by converting the values to categorical values.

Model 1: Build a neural network without convolutional layers to do the classification task

Step 1: Defining a Model Architecture

- We start by defining a sequential model. A Sequential model is a linear stack of layers in which the layers are added one by one, through which the data flows sequentially.

```
model_T1 = Sequential()
```

- Input Layer: Then we add our first layer with input data, where a 2D image of 28*28 pixels is flattened into a 1D array of 784 values.

```
model_T1.add(Flatten(input_shape=(28, 28, 1)))
```

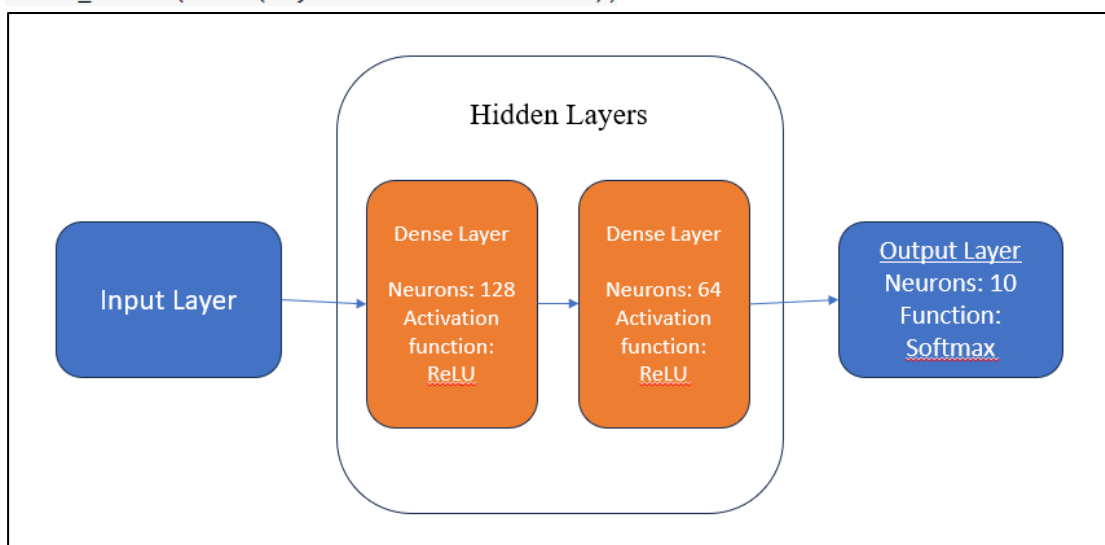
- Two hidden Layers: Then we add two dense layers one by one. The first layer has 128 neurons with 'ReLU' as an activation function. In simple words, 'ReLU' function outputs the input value if positive and zero otherwise. The second layer has 64 neurons with 'ReLU' as an activation function.

```
model_T1.add(Dense(128, activation='relu'))
```

```
model_T1.add(Dense(64, activation='relu'))
```

- Output Layer: The final layer has the output with 10 neurons with the probability of the input image belonging to each of the 10 possible classes. The activation function 'softmax' is used which converts the raw output values into probabilities. We used 'softmax' because it is a multi-class classification problem.

```
model_T1.add(Dense(10, activation='softmax'))
```



Model: "sequential_1"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_1 (Dense)	(None, 128)	100480
dense_2 (Dense)	(None, 64)	8256
dense_3 (Dense)	(None, 10)	650
Total params: 109,386		
Trainable params: 109,386		
Non-trainable params: 0		

Step 2: Compiling the model

In this step, we define the Optimizer, Loss function and evaluation metrics.

For this, 'categorical_crossentropy' is taken as the loss function, which measures the dissimilarity between the predicted and actual distribution. The optimizer is used for updating the parameters of the model i.e. weights and biases so that the loss function is minimized. 'Adam' is used as an optimizer with the default settings.

Step 3: Training the model

Random state is set to 1, so that comparing evaluation metrics is possible

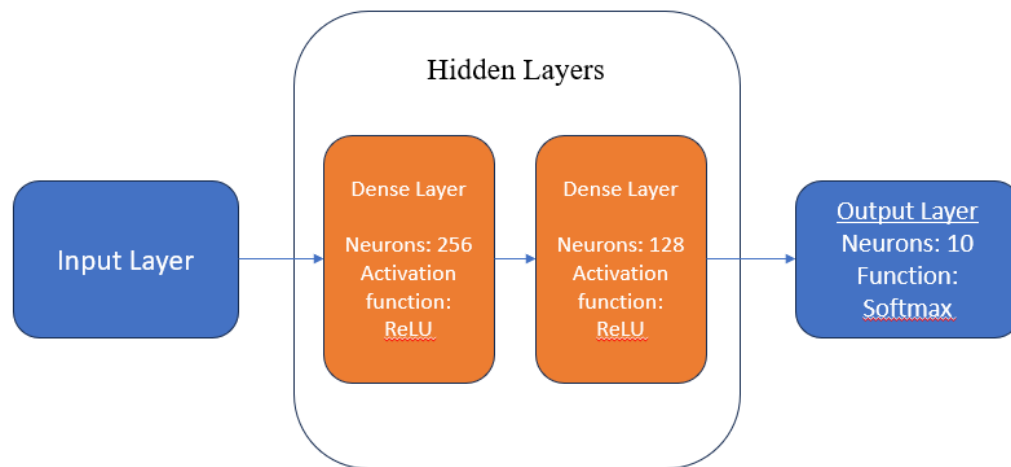
In this step, we train the model with 10 epochs initially. An epoch can be described as one complete pass through the entire training dataset. As suggested by the assignment requirements, the model is trained without any separate validation set, but test set is used to monitor the training process.

Step 4: Evaluating the model with Testing Dataset

This shows that our model has an accuracy of 87.66%.

Step 5: Changing the model structure to get an improved performance

Changing the number of neurons: Changed the number of neurons of hidden layers to 256 and 128 respectively, with no increase in number of layers.



This change showed an improved model accuracy of 88.64%.

Model 2: Build a neural network with the use of convolutional layers

Step 1: Defining the Model Architecture

- We start by defining a sequential model. A Sequential model is a linear stack of layers in which the layers are added one by one, through which the data flows sequentially.

```
model_T2 = Sequential()
```

- Convolutional Layer 1: Convolutional layers close to input learn low-level features. As this is the first Convolutional layer it will have low level features. I have taken this layer to have 32 filters (3*3 kernel), which means a total of $32 \times 3 \times 3 = 288$ parameters. The activation function used is 'ReLU', which introduces non-linearity.

```
model_T2.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
```

- Pooling Layer 1: This layer is used to achieve down sampling, which means to create a lower resolution of an input while retaining the important features. This reduces the size of each feature map by a factor of 2. Max pooling is used, which takes the maximum value for each patch of the feature map.

```
model_T2.add(MaxPooling2D((2, 2)))
```

- Convolutional Layer 2: This layer has 64 filters with 'ReLU' as activation function.

```
model_T2.add(Conv2D(64, (3, 3), activation='relu'))
```

- Pooling Layer 2: This layer is again using Max pooling to down sample by a factor of 2.

```
model_T2.add(MaxPooling2D((2, 2)))
```

- Flatten Layer: After the above layers a flatten layer is added, which converts feature maps into a 1D vector, which is required before transitioning to the fully connected layers.

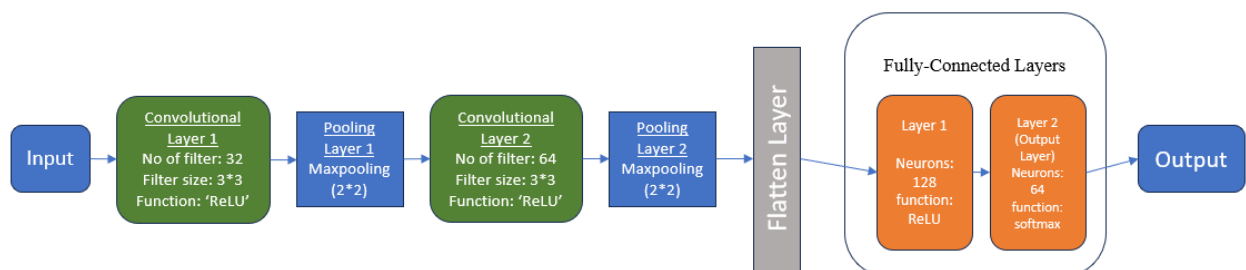
```
model_T2.add(Flatten())
```

- Fully Connected Layer 1: A dense layer with 'ReLU' as a nonlinear activation function to create combinations of features for final predictions. This is made of 128 neurons.

```
model_T2.add(Dense(128, activation='relu'))
```

- Fully Connected Layer 2 (Output Layer): This is the final layer with 10 neurons corresponding to the 10 classes for prediction. The activation function used is 'softmax' which assigns probability to each class.

```
model_T2.add(Dense(10, activation='softmax'))
```



Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_2 (Conv2D)	(None, 14, 14, 64)	18496
max_pooling2d_2 (MaxPooling2D)	(None, 7, 7, 64)	0
flatten_3 (Flatten)	(None, 3136)	0
dense_7 (Dense)	(None, 128)	401536
dense_8 (Dense)	(None, 10)	1290
Total params: 421,642		
Trainable params: 421,642		
Non-trainable params: 0		

Step 2: Compiling the model

In this step, we define the Optimizer, Loss function and evaluation metrics.

For this, 'categorical_crossentropy' is taken as the loss function, which measures the dissimilarity between the predicted and actual distribution. The optimizer is used for updating the parameters of the model i.e. weights and biases so that the loss function is minimized.

'Adam' is used as an optimizer. Generally, Adam adapts the learning rates of each parameter individually, which can lead to faster convergence. But here we use it with a custom learning rate of 0.002.

Step 3: Training the model

Random state is set to 1, so that comparing evaluation metrics is possible

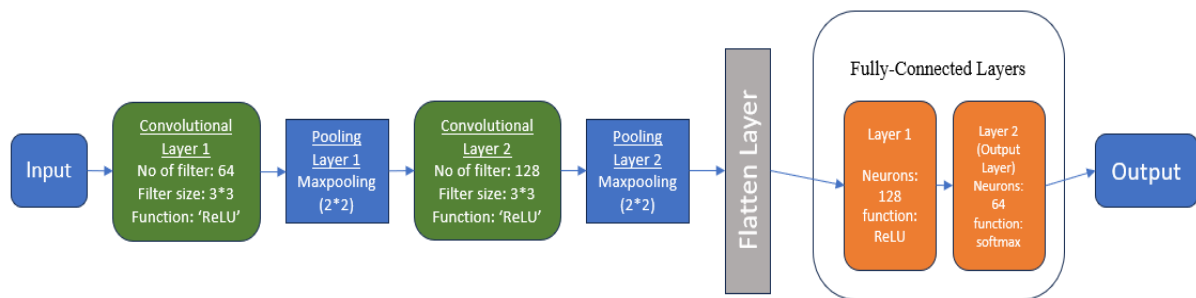
In this step, we train the model with 10 epochs initially. An epoch can be described as one complete pass through the entire training dataset. As suggested by the assignment requirements, the model is trained without any separate validation set, but test set is used to monitor the training process.

Step 4: Evaluating the model with Testing Dataset

This shows that our model has an accuracy of 90.71%.

Step 5: Changing the model structure to get an improved performance

Changing the model structure by increasing the number of filters in the convolutional layer to 64 and 128 respectively.



This change showed an improved model accuracy of 90.80%.

Model 3: Changing the type of optimizer or learning rate and see how these changes can influence model performance

To perform this task, the optimizer is set to 'sgd' with a custom setting of changing learning rate of 0.002, 0.003, 0.004 and 0.008.

The performance results are the following:

Learning Rate	Accuracy
0.002	91.92%
0.004	92.02%
0.006	91.99%
0.008	91.78%

Based on this we can say that the Learning rate of 0.004 provides the best accuracy.

Comparing the results

The following are ranked in descending order of their accuracy performance.

Task	Brief Model Description	Optimizer	Accuracy	Rank
Task 3	With 2 Convolutional Layers (64 and 128 filters)	Custom SGD	92.02% (lr = 0.004)	1
Task 2	With 2 Convolutional Layers (64 and 128 filters)	Custom Adam	90.80%	2
Task 2	With 2 Convolutional Layers (32 and 64 filters)	Custom Adam	90.71%	3
Task 1	Without Convolutional Layers (256 and 128 neurons)	Adam	88.64%	4
Task 1	Without Convolutional Layers (128 and 64 neurons)	Adam	87.66%	5

In the above table, only the best learning rate from Task 3 is considered.

Based on the whole analysis, the use of Convolutional Networks give a better performance results than without them. Also, the 'sgd' optimizer seems to be better fitted for this dataset.